RETO 2 · DISEÑO DE SERVICIOS

RapiEntrega

Plataforma de entregas a domicilio

Diseño SOA, contratos REST e implementación demostrativa con Spring Boot

CATÁLOGO • PEDIDOS • PAGOS • LOGÍSTICA • SEGUIMIENTO • FACTURACIÓN

Datos de entrega	Información
Asignatura	Arquitectura Orientada a Servicios (SOA) · Unidad 2
Reto	Reto 2: Plataforma de entregas a domicilio
Integrantes	_____
Docente / grupo	_____
Fecha	_____

Documento de diseño de 10–15 páginas acompañado de código fuente, diagramas, contratos API y pruebas de integración.

01 · CONTEXTO

Resumen ejecutivo y alcance

RapiEntrega conecta tres actores operativos -cliente, restaurante y repartidor- para completar un pedido de comida de principio a fin. El caso de la Unidad 2 incorpora dos dependencias no controladas por la plataforma: la pasarela externa de tarjetas y el sistema de facturación heredado de 2012, que no puede reemplazarse.

Problema y respuesta de diseño

Necesidad del caso	Decisión de diseño	Resultado esperado
Catálogo consultable y pedido correcto	Servicio de catálogo como fuente de nombres/precios; el cliente envía identificadores y cantidades.	No confiar el precio al cliente; validar pertenencia de productos al restaurante.
Pago externo	Puerto `PasarelaPagos` y adaptador simulado.	Aislar el proveedor y no recibir datos de tarjeta en la API.
Aceptar en cinco minutos	Estado `PENDIENTE_ACEPTACION`, marca de tiempo y tarea programada.	Si el restaurante rechaza o vence el plazo, cancelar y compensar con reembolso.
Repartidor cercano y tracking	Asignador por distancia Haversine + publicación de eventos + SSE.	Actualizar estado/ubicación sin bloquear cada operación del pedido.
Facturación no reemplazable	`SistemaFacturacionLegadoPort` y `SistemaFacturacionLegadoAdapter`.	Conservar el límite con el legado; sustituir el simulador por un conector real sin cambiar el servicio consumidor.

Alcance de la implementación

Se incluye una API REST versionada en `/api/v1`, DTO validados, modelos de dominio, servicios y repositorios, eventos internos, seguimiento SSE, manejo uniforme de errores, OpenAPI y pruebas MockMvc. Los datos iniciales se cargan en repositorios en memoria con restaurantes, productos y repartidores de demostración.

Límite importante: la ejecución local demuestra el flujo de integración, no opera pagos reales, no se conecta al sistema fiscal heredado y no persiste datos después de reiniciar. El diseño señala las sustituciones necesarias para producción.

Criterios de aceptación vinculados a la rúbrica

- Modelar el proceso con cinco pasos explícitos y resultados verificables.
- Trazar actividades hacia capacidades y dominios sin asignar la misma regla a varios servicios propietarios.
- Justificar al menos tres patrones nombrados desde el catálogo de Thomas Erl.
- Diferenciar request-response síncrono, pub/sub asíncrono y streaming SSE; especificar cuatro capacidades completas.

02 · MODELADO

Proceso de modelado de servicios en cinco pasos

El PDF del reto exige aplicar cinco pasos, pero no enumera sus nombres. Para que la selección sea trazable, este documento adopta como marco los primeros cinco pasos del modelado de servicios REST descritos por Thomas Erl: descomponer el proceso, filtrar acciones inadecuadas, identificar candidatos de servicios de entidad, aislar la lógica específica del proceso e identificar recursos [2].

Paso	Aplicación a “Pedido de comida”	Resultado verificable
1. Descomponer el proceso	Consultar restaurantes/menú; seleccionar productos; validar; crear pedido; pagar; esperar decisión; aceptar/rechazar; asignar repartidor; publicar ubicación; confirmar entrega; calificar; solicitar factura.	Secuencia granular y escenarios alternos en diagrama de proceso.
2. Filtrar acciones inadecuadas	“Dar clic”, mostrar pantalla y otros detalles UI no son capacidades de negocio. “Esperar cinco minutos” se modela como política/temporizador, no como endpoint invocado por el usuario.	Catálogo depurado de acciones con propósito de negocio.
3. Definir candidatos de entidad	Restaurante, Producto, Pedido, DetallePedido, Repartidor, Calificación y Factura. Sus contratos exponen datos controlados y operaciones de consulta/actualización.	Recursos `/restaurantes`, `/productos`, `/pedidos`, `/repartidores`, `/calificacion`, `/facturas`.
4. Identificar lógica específica del proceso	Calcular total; coordinar pago y decisión del restaurante; aplicar ventana de cinco minutos; compensar el pago; reservar liberación del repartidor.	Servicio de proceso `PedidoService` coordina capacidades, no duplica el catálogo ni la pasarela.
5. Identificar recursos REST	Definir URI, métodos HTTP, JSON, códigos de respuesta, estados y relaciones entre los recursos.	API versionada `/api/v1`, contratos con DTO, validación y OpenAPI.

Reglas de descomposición

- Cada servicio tiene una responsabilidad principal y un contrato explícito: catálogo, pedidos, pagos, logística, seguimiento, reputación y facturación.
- El total del pedido se deriva de los precios del catálogo; no se acepta un importe enviado por la app del cliente.
- Las acciones de negocio que pueden fallar en proveedores externos tienen resultados/errores explícitos y adaptadores sustituibles.

Resultado del modelado: una composición con `PedidoService` como coordinador del proceso, y capacidades desacopladas para catálogo, pago, asignación, tracking, calificación y factura.

03 · MAPEO

Matriz de actividades y servicios

La matriz establece una sola responsabilidad principal por actividad. Las llamadas cruzadas son dependencias explícitas; no significan que todos los servicios compartan sus repositorios. La implementación actual empaqueta las clases en una sola aplicación para simplificar la práctica, pero los contratos permiten separarlas en despliegues independientes más adelante.

#	Actividad de negocio	Operación / capacidad candidata	Servicio propietario	Dominio
1	Buscar restaurantes abiertos	`listarRestaurantes()`	RestauranteService	Catálogo
2	Consultar productos y precios	`listarProductos(restauranteId)`	RestauranteService	Catálogo
3	Crear pedido y calcular importe	`crearPedido(request)`	PedidoService	Pedidos
4	Consultar líneas y estado	`consultarPedido(id)` / `listarDetalles(id)`	PedidoService / DetallePedidoController	Pedidos
5	Autorizar pago	`procesarPago(pedidoId, token)`	PagoService vía PedidoService	Pagos
6	Aceptar o rechazar pedido	`decidirPedidoRestaurante(id, decision)`	PedidoService	Pedidos
7	Cancelar por plazo vencido y reembolsar	`expirarPedidosPendientes()` / `reembolsarPago()`	PedidoService + PagoService	Pedidos / Pagos
8	Elegir repartidor cercano	`asignarRepartidor(id)`	AsignacionRepartidorService	Logística
9	Publicar ubicación	`actualizarUbicacion(id, repartidorId, lat, lon)`	AsignacionRepartidorService	Logística
10	Distribuir cambio de estado/ubicación	`publicarEventoPedido(evento)` / `suscribirEventos(id)`	EventoPedidoService / SeguimientoService	Seguimiento
11	Confirmar entrega	`confirmarEntrega(id, repartidorId)`	AsignacionRepartidorService	Logística
12	Calificar entrega	`calificarPedido(id, puntuacion, comentario)`	CalificacionService	Reputación
13	Emitir factura en legado	`solicitarFactura(pedidoId, datosFiscales)`	FacturaService + Legacy Adapter	Facturación

Operaciones convertidas a recursos y verbos HTTP

Las consultas usan GET; la creación y los cambios de estado explícitos usan POST porque ejecutan comandos del proceso; la API no expone un endpoint público para forzar el vencimiento del temporizador. El sistema programa automáticamente la revisión. En una evolución REST más estricta podrían representarse algunas decisiones como PATCH de un recurso de pedido, manteniendo el contrato de transición de estado.

04 · DOMINIOS

Delimitación de dominios y servicios

Dominio	Servicios/capacidades	Datos que controla	Fuera de su responsabilidad
1. Catálogo y restaurantes	RestauranteService, RestauranteController, ProductoController	Restaurantes abiertos, menú, precio, disponibilidad.	No crea pedidos, no cobra y no asigna repartidores.
2. Pedidos / orquestación	PedidoService, PedidoController, DetallePedidoController	Identificador, cliente, líneas, importes calculados, estado y plazo de aceptación.	No almacena tarjetas ni implementa reglas internas del sistema legado.
3. Pagos	PagoService, `PasarelaPagos`	Referencia de cobro/reembolso y estado de resultado.	No decide el menú ni la ubicación del repartidor.
4. Logística y reparto	AsignacionRepartidorService, EntregaController, RepartidorRepository	Disponibilidad, asignación, coordenadas y confirmación de entrega.	No cambia el catálogo ni calcula la factura fiscal.
5. Seguimiento/eventos	EventoPedidoService, SeguimientoService, SSE	Eventos con id de pedido, tipo, estado, mensaje y timestamp.	No es fuente principal del estado; el pedido conserva el estado actual.
6. Reputación / cliente	ClienteService, ClienteController, CalificacionService	Consulta de pedidos por correo de demostración y calificación única por pedido.	No reemplaza identidad/autenticación real.
7. Facturación	FacturaService, `SistemaFacturacionLegadoAdapter`	Solicitud, respuesta e identificador de factura.	No reemplaza al sistema de 2012; solo adapta el contrato externo.

Límite del despliegue de esta entrega

Los siete dominios son límites lógicos dentro de una aplicación Spring Boot modular (monolito modular), no siete microservicios desplegados. Esta decisión reduce fricción de arranque en una práctica académica y conserva contratos y responsabilidades separados. Al escalar, se extraen servicios por carga y autonomía operativa, moviendo el estado a almacenamiento persistente y publicando eventos por broker durable.

Evitar solapamientos: PedidoService es dueño de la transición del proceso; PagoService encapsula el puerto de pagos; AsignacionRepartidorService asigna y actualiza ubicación; SeguimientoService retransmite eventos sin convertirse en el dueño de la verdad del pedido.

05 · ARQUITECTURA

Diagrama de dominios e interacciones

Las flechas muestran el sentido principal de las solicitudes y notificaciones. Las líneas discontinuas representan llamadas internas o eventos del proceso; el color cálido identifica dependencias externas/legadas. Se evita la integración directa del cliente con los proveedores externos: todos los contratos pasan por una capacidad de la plataforma.

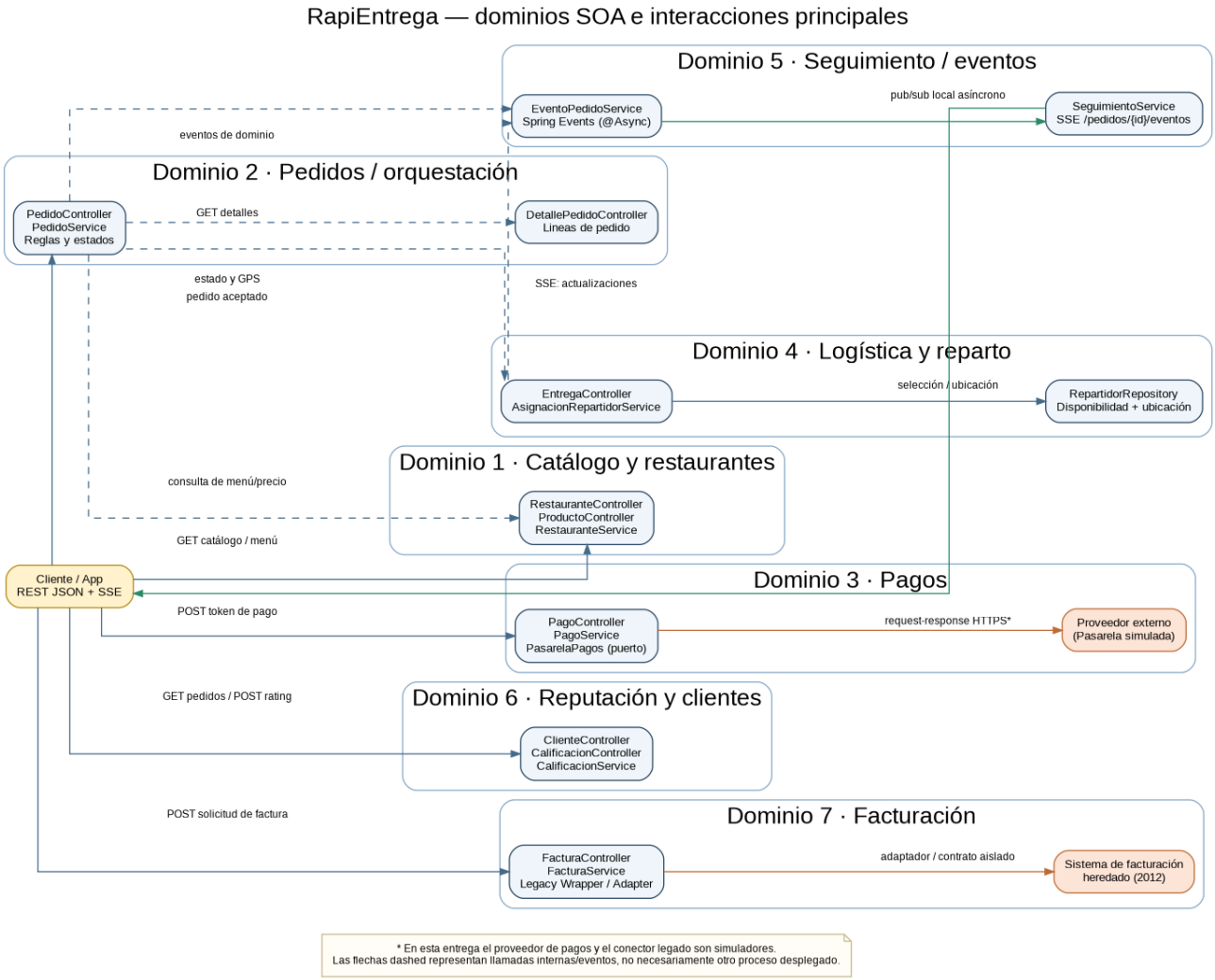


Figura 1. Dominios lógicos de RapiEntrega. La implementación empaquetada no implica que los dominios deban desplegarse juntos.

Contratos que atraviesan los límites

- Pedido: `id`, `estado`, `estadoPago`, `restauranteId`, líneas, total, marcas de tiempo, repartidor asignado y última ubicación disponible.
- Evento de pedido: `pedidoId`, `tipo`, `estado`, `mensaje`, `fecha`; los consumidores pueden reaccionar sin bloquear la operación que originó el cambio.
- Pago: `pedidoId`, importe calculado en el servidor y token de proveedor; respuesta con `aprobado`, `referencia` y `mensaje`.
- Factura: `pedidoId`, RFC, razón social, correo; respuesta aislada con id externo, total, fecha y estado del adaptador legado.

06 · PATRONES

Patrones SOA elegidos y justificación

Se usan nombres del catálogo de Thomas Erl y se concreta cada uno con el problema específico del caso. La lista exacta evaluada en clase puede variar; el PDF sugiere Erl (2009), pero no incluye la presentación del docente ni un catálogo interno, por lo que se documenta la referencia bibliográfica empleada.

Patrón (catálogo Erl)	Problema del caso	Aplicación en RapiEntrega	Efecto y compromiso
Process Abstraction	El cliente no debe coordinar llamadas ni comprender todas las transiciones de pedido.	`PedidoService` encapsula el proceso y sus reglas; los controllers traducen HTTP a operaciones.	Coordinación central y comportamiento coherente; se debe evitar un servicio monolítico que absorba capacidades ajenas.
Entity Abstraction	Hay recursos con identidad y ciclo de vida diferente.	`/restaurantes`, `/pedidos`, `/repartidores`, `/facturas` y `/calificacion` representan entidades.	Contratos más descubribles; no confundir entidad de negocio con un único CRUD sin reglas.
Service Façade	No exponer detalles de proveedor y lógica interna a cada consumidor.	`PagoService` y `FacturaService` esconden adaptadores; los controllers ofrecen un punto HTTP estable.	Reduce acoplamiento; la fachada debe mantener errores y estados explícitos.
Legacy Wrapper	El sistema de facturación 2012 no puede reemplazarse.	`SistemaFacturacionLegadoPort` + `SistemaFacturacionLegadoAdapter` aísla el formato/protocolo heredado.	Permite cambiar el conector sin alterar los consumidores; actualmente es simulación, no integración real.
Event-Driven Messaging	El seguimiento no debe obligar al flujo principal a esperar a cada consumidor.	`EventoPedidoService` publica eventos; `SeguimientoService` escucha asíncronamente y los transmite por SSE.	Productor y consumidor quedan menos acoplados; los eventos locales no son durables.
Compensating Service Transaction	Un pedido pagado puede ser rechazado o expirar, por lo que hay que deshacer el efecto financiero.	`cancelarYReembolsar` registra cancelación y solicita reembolso por el puerto de pagos.	Modela una compensación distribuida, no una transacción ACID; producción requiere idempotencia y conciliación.
Asynchronous Queuing (evolución)	Un evento no debe perderse si se reinicia un nodo o si el consumidor está caído.	No se afirma una cola durable implementada; se recomienda RabbitMQ/Kafka entre publicador y consumidores en producción.	Retención, reintentos y back-pressure; añade infraestructura y operación.

Por qué no introducir un ESB completo en esta práctica

El número de flujos y sistemas externos de la demostración no justifica un bus empresarial como requisito de ejecución. Se define el contrato de eventos para que un broker se incorpore después, sin confundir el `ApplicationEventPublisher` en proceso con mensajería distribuida durable.

07 · COMUNICACIÓN

Interacciones, mensajes y publicación-suscripción

Interacción	Estilo	Patrón / transporte	Motivo de la elección
Cliente → Catálogo	Síncrona	REST GET, request-response	La pantalla necesita menú y precios para continuar; respuesta corta e inmediata.
Cliente → Pedidos	Síncrona	REST POST/GET + JSON	La creación debe devolver id, importes y estado; consulta directa de estado actual.
Pedido → Pasarela	Síncrona en prototipo	Request-response mediante puerto/adaptador	El flujo necesita saber si el pago fue aprobado antes de iniciar la ventana de aceptación. Timeout y reintento idempotente son requisitos de producción.
Pedido → Restaurante	Síncrona como operación de decisión; temporizador local	POST de decisión; scheduler para expiración	La aceptación es respuesta explícita; la expiración no depende de que el cliente consulte.
Pedido / Logística → Eventos	Asíncrona local	Event-Driven Messaging, pub/sub Spring Events + `@Async`	Varios consumidores potenciales reciben el cambio sin llamadas punto a punto del productor.
Seguimiento → App cliente	Asíncrona / stream unidireccional	SSE `text/event-stream`	Push de cambios de estado/ubicación sin consulta repetitiva; no se necesita canal cliente-servidor bidireccional.
Pedido → Asignación	Síncrona interna	Llamada de servicio / composición	Debe conocer si se reservó un repartidor; en versión distribuida usar comando con respuesta o proceso asíncrono.
FacturaService → Legado	Síncrona en adaptador simulado	Request-response, `Legacy Wrapper`	Se encapsula la interfaz no reemplazable y se traduce su resultado al modelo de la plataforma.
Cliente → Calificación	Síncrona	REST POST, validación 1–5	La confirmación permite detectar si la calificación fue aceptada o ya existía.

Dónde se usa pub/sub y cómo migraría

El evento `PedidoActualizadoEvent` se publica cuando cambia el estado o hay una ubicación nueva. En el prototipo, Spring Events y `@Async` desacoplan el productor del suscriptor SSE dentro del mismo proceso. Para varios nodos, retención de eventos, reintentos y recuperación tras caída, el paso de producción es publicar un contrato versionado a un broker durable; cada consumidor se suscribe a los tipos que necesita.

08 · ESPECIFICACIÓN 1 DE 4

crearPedido

Registrar un pedido desde un restaurante abierto y devolver un total calculado por el servidor.

Elemento	Especificación
Operación / servicio	PedidoService / PedidoController
Método y recurso	POST /api/v1/pedidos
Objetivo	Registrar un pedido desde un restaurante abierto y devolver un total calculado por el servidor.

Entrada (contrato y tipos)

Campo	Tipo / formato	Restricción
clienteNombre	String	Obligatorio, no vacío.
clienteCorreo	String (email)	Obligatorio, formato email.
restauranteId	String	Obligatorio; restaurante existente y abierto.
direccionEntrega	String	Obligatorio, no vacío.
latitudEntrega	Double	Obligatorio; -90 a 90.
longitudEntrega	Double	Obligatorio; -180 a 180.
productos	Array de objetos	Al menos uno.
productos[].productoId	String	Obligatorio; debe pertenecer al restaurante.
productos[].cantidad	Integer	Mínimo 1.

Salida

HTTP 201 Created con un `PedidoModel`: id asignado, restaurante, líneas (precio unitario y subtotal), subtotal, tarifa de envío de demostración de $29.00, total, `PENDIENTE_PAGO`, `PENDIENTE` y fecha. Los importes se calculan con `BigDecimal` y dos decimales.

Reglas de negocio

- El precio recibido desde el móvil no es parte del contrato: la fuente de verdad es el menú del restaurante.
- No se permite un producto de otro restaurante o marcado como no disponible.
- El pedido recién creado no implica autorización ni captura del pago.
- Se emite `PEDIDO_CREADO` para los consumidores de seguimiento; el evento no sustituye el recurso de pedido.

Errores y códigos HTTP

- 400 Bad Request: campo inválido, cantidad menor que uno o JSON mal formado.
- 404 Not Found: restaurante/producto inexistente o producto que no corresponde al restaurante.
- 409 Conflict: restaurante cerrado o producto no disponible.
- 500 Internal Server Error: fallo no previsto, con mensaje genérico al cliente.

Calidad de servicio (objetivo de diseño; no medido)

Objetivo propuesto: p95 inferior a 500 ms para operación interna sin proveedor externo, disponibilidad del servicio ≥99.5% mensual en despliegue productivo, respuesta JSON versionada y trazabilidad por `pedidoId`. Estos objetivos necesitan pruebas y observabilidad; no se reportan como métricas medidas.

Implementación: Clase `PedidoService.crear`, DTO `CrearPedidoRequest`, `ItemPedidoRequest`, `RestauranteRepository` y `PedidoRepository`. En esta entrega ambos repositorios son mapas concurrentes de demo.

08 · ESPECIFICACIÓN 2 DE 4

confirmarPago

Solicitar autorización de pago y, si es positiva, iniciar la ventana de aceptación del restaurante.

Elemento	Especificación
Operación / servicio	PagoService (coordinado por PedidoService) / PagoController
Método y recurso	POST /api/v1/pedidos/{pedidoId}/pago
Objetivo	Solicitar autorización de pago y, si es positiva, iniciar la ventana de aceptación del restaurante.

Entrada (contrato y tipos)

Campo	Tipo / formato	Restricción
pedidoId	String en URI	Debe existir y estar pendiente de pago.
tokenPago	String	Obligatorio; token de proveedor, nunca PAN/CVV.
monto	No se envía	Lo toma del pedido calculado por el servidor.
resultado	ResultadoPagoModel	aprobado, referencia, mensaje del adaptador.

Salida

HTTP 200 OK con pedido actualizado. Pago aprobado: `estadoPago=APROBADO`, referencia y fecha de pago, `estado=PENDIENTE_ACEPTACION`. Pago rechazado: `estadoPago=RECHAZADO` y `estado=CANCELADO`. Evento `PAGO_CONFIRMADO` o `PAGO_RECHAZADO`.

Reglas de negocio

- La capacidad solo opera en `PENDIENTE_PAGO`; llamadas repetidas no deben producir otro cobro. El prototipo rechaza la repetición por transición de estado; producción requiere clave de idempotencia conciliada con la pasarela.
- El importe y moneda se derivan del pedido; el navegador/móvil no puede modificarlos.
- El proveedor se invoca a través de la interfaz `PasarelaPagos`; no se acopla `PedidoService` al proveedor concreto.
- La ventana de cinco minutos comienza al confirmar la aprobación del pago.

Errores y códigos HTTP

- 400: falta token o petición mal formada.
- 404: pedido no encontrado.
- 409: estado no permite pago.
- 200 con `CANCELADO` y `RECHAZADO`: declinación de pago conocida, no error técnico HTTP.
- 5xx: fallo no previsto de la integración; producción debe manejar timeout e incertidumbre sin duplicar cobros.

Calidad de servicio (objetivo de diseño; no medido)

Objetivo propuesto: evitar cobro duplicado mediante idempotencia por pedido + clave de operación; timeout del proveedor configurable; tokenización PCI del lado de la pasarela; no almacenar ni registrar datos de tarjeta. La latencia de la pasarela queda fuera del objetivo interno p95.

Implementación: La interfaz `PasarelaPagos` y `PasarelaPagosSimulada` generan referencias sintéticas. `tok_demo_rechazado` permite probar el camino de rechazo sin afectar dinero real.

08 · ESPECIFICACIÓN 3 DE 4

decidirPedidoRestaurante

Registrar aceptación o rechazo mientras el pedido se encuentre dentro de la ventana de cinco minutos.

Elemento	Especificación
Operación / servicio	PedidoService / EntregaController
Método y recurso	POST /api/v1/pedidos/{pedidoId}/decision-restaurante
Objetivo	Registrar aceptación o rechazo mientras el pedido se encuentre dentro de la ventana de cinco minutos.

Entrada (contrato y tipos)

Campo	Tipo / formato	Restricción
pedidoId	String en URI	Debe existir; estado pendiente de aceptación.
aceptado	Boolean	Obligatorio.
motivo	String nullable	Recomendado cuando se rechaza.
pagadoEn	Instant interno	Marca de tiempo usada para validar el plazo.
plazo	5 minutos	Regla fija del caso, desde el pago aprobado.

Salida

HTTP 200 OK con el pedido actualizado. Si acepta en plazo: `ACEPTADO`. Si rechaza: `CANCELADO`, razón y `estadoPago=REEMBOLSADO` tras compensación simulada. Si el plazo ya terminó: cancelación y reembolso en vez de aceptar.

Reglas de negocio

- Solo el estado `PENDIENTE_ACEPTACION` admite decisión.
- El plazo se calcula desde la aprobación del pago, no desde la creación del carrito.
- El scheduler inspecciona pedidos vencidos cada 10 segundos por defecto; si el restaurante no respondió, cancela y solicita reembolso.
- La actualización y la expiración se sincronizan por pedido en el prototipo (`synchronized`) para disminuir la carrera entre aceptación y cancelación.
- En producción la autorización del restaurante debe comprobarse con identidad y roles.

Errores y códigos HTTP

- 400: `aceptado` ausente o JSON inválido.
- 404: pedido inexistente.
- 409: ya fue aceptado/cancelado o estado incorrecto.
- 200 con estado `CANCELADO`: expiración vencida procesada durante el intento.
- Si falla el reembolso, se eleva conflicto para compensación manual; el caso debe generar alerta operativa.

Calidad de servicio (objetivo de diseño; no medido)

Objetivo: respetar el límite de cinco minutos con un margen de escaneo de hasta 10 segundos en el prototipo. Para producción, usar un scheduler distribuido o evento con deadline, transiciones condicionales atómicas, registro de auditoría, alarma y reconciliación de reembolsos.

Implementación: `PedidoService.decidirRestaurante`, `cancelarPedidosVencidos` y `PedidosTimeoutScheduler`. La compensación se aplica mediante `PagoService.reembolsar`; la ventana es lógica de servidor.

08 · ESPECIFICACIÓN 4 DE 4

asignarRepartidor

Reservar el repartidor disponible más cercano al restaurante para un pedido aceptado.

Elemento	Especificación
Operación / servicio	AsignacionRepartidorService / EntregaController
Método y recurso	POST /api/v1/pedidos/{pedidoId}/asignacion
Objetivo	Reservar el repartidor disponible más cercano al restaurante para un pedido aceptado.

Entrada (contrato y tipos)

Campo	Tipo / formato	Restricción
pedidoId	String en URI	Pedido debe estar `ACEPTADO`.
ubicación del restaurante	latitud/longitud Double	Catálogo; no se recibe desde cliente.
repartidores	id, latitud, longitud, disponible	Sólo candidatos disponibles.
criterio	Distancia Haversine	Menor distancia geográfica al restaurante.
respuesta	PedidoModel	Incluye id/nombre de repartidor y `REPARTIDOR_ASIGNADO`.

Salida

HTTP 200 OK con el identificador/nombre del repartidor, pedido en `REPARTIDOR_ASIGNADO` y evento `REPARTIDOR_ASIGNADO`. El repartidor elegido se marca no disponible para reducir asignaciones simultáneas.

Reglas de negocio

- Sólo pedidos aceptados pueden asignarse.
- Se compara la ubicación del repartidor con la ubicación del restaurante (punto de recogida), no con la del cliente.
- Si no hay candidatos disponibles, no se asigna y se devuelve 409; el pedido se mantiene aceptado para permitir un reintento.
- La ubicación del repartidor se actualiza mediante el endpoint de entrega; el estado pasa a `EN_CAMINO` y el evento se publica a suscriptores.
- La confirmación de entrega libera al repartidor y permite que el cliente califique el servicio.

Errores y códigos HTTP

- 404: pedido o restaurante inexistente.
- 409: estado del pedido incorrecto, no hay repartidores disponibles o repartidor no corresponde al pedido.
- 500: error no previsto. En despliegue distribuido, reservar debe ser una operación atómica con lease/timeout.

Calidad de servicio (objetivo de diseño; no medido)

Objetivo propuesto: selección interna p95 inferior a 300 ms con hasta 1,000 candidatos en memoria de servicio; consistencia de una única asignación activa por pedido; coordenadas validadas y canal TLS en producción. La capacidad de escala depende de dividir carga, persistir disponibilidad y resolver concurrencia.

Implementación: `AsignacionRepartidorService` ejecuta Haversine sobre candidatos disponibles y publica el resultado. Para producción, usar un índice geoespacial y reserva atómica, no un escaneo lineal en repositorio distribuido.

09 · CONTRATOS

Superficie REST, estados y códigos de respuesta

Método	Recurso	Contrato / resultado clave
GET	`/api/v1/restaurantes`	Lista restaurantes abiertos con menú.
GET	`/api/v1/restaurantes/{id}`	Restaurante y su menú.
GET	`/api/v1/restaurantes/{id}/productos`	Productos disponibles y precio de catálogo.
POST	`/api/v1/pedidos`	`CrearPedidoRequest` → 201 y pedido en PENDIENTE_PAGO.
GET	`/api/v1/pedidos/{id}`	Estado actual, total, repartidor y última ubicación.
GET	`/api/v1/pedidos/{id}/detalles`	Líneas: producto, cantidad, unitario y subtotal.
POST	`/api/v1/pedidos/{id}/pago`	Token → estado pago / inicio de plazo.
POST	`/api/v1/pedidos/{id}/decision-restaurante`	`{aceptado, motivo}` → aceptación o cancelación compensada.
POST	`/api/v1/pedidos/{id}/asignacion`	Asigna el repartidor más próximo disponible.
POST	`/api/v1/pedidos/{id}/ubicacion`	`{repartidorId, latitud, longitud}` → estado EN_CAMINO + evento.
GET	`/api/v1/pedidos/{id}/eventos`	SSE: `PEDIDO_CREADO`, `PAGO_CONFIRMADO`, `UBICACION_ACTUALIZADA`, etc.
POST	`/api/v1/pedidos/{id}/entrega`	Confirma entrega y libera al repartidor.
POST	`/api/v1/pedidos/{id}/calificacion`	`{puntuacion:1..5, comentario}`; sólo una vez.
POST	`/api/v1/facturas`	Solicita factura vía adaptador legado simulado.

Política de errores

Todas las excepciones controladas se normalizan en `ApiError`: `timestamp`, `status`, `error`, `message`, `path` y `details`. Se utilizan 400 para validación, 404 para recursos inexistentes y 409 para transiciones no válidas/reglas de negocio. Un rechazo de pago conocido se representa en el estado del pedido, mientras que un fallo técnico real de la pasarela debe distinguirse de una declinación de negocio.

Seguridad y datos personales

- No pedir ni almacenar PAN o CVV; intercambiar token seguro del proveedor. Las llamadas reales deben usar TLS y gestionar secretos en un vault o variables de entorno protegidas.
- La API de demostración no implementa login, OAuth2/JWT, autorización por rol, rate limiting ni anonimización; el endpoint de pedidos por correo no es seguro para producción y sirve sólo como ejemplo didáctico.
- Validar coordenadas, cantidades, estado de transición e identificador de repartidor; auditar decisiones, cancelaciones, reembolsos y solicitudes de factura.

10 · CALIDAD

Escalabilidad, confiabilidad y plan de pruebas

Objetivos no funcionales propuestos

Atributo	Objetivo de diseño	Cómo validarlo / evolución
Escalabilidad	Controllers sin estado de sesión; servicios extraíbles por dominio.	Prueba de carga; escalar réplicas, compartir base persistente y externalizar eventos.
Disponibilidad	99.5% mensual como meta académica inicial.	Monitoreo, health checks, alertas y despliegue redundante; no está medido en esta versión.
Latencia	p95 <500 ms para operaciones internas sin proveedor.	Prueba JMeter, métricas separadas por endpoint y distribución de latencia.
Consistencia	Una decisión válida por pedido; una asignación activa; un reembolso compensatorio.	Pruebas concurrentes; CAS/versión en base de datos; claves idempotentes.
Observabilidad	Correlación por pedido, logs estructurados y métricas de errores.	Añadir traceId, OpenTelemetry, dashboards y alertas por timeout/reembolso fallido.
Eventos	Pub/sub sin bloquear el productor, seguimiento por SSE.	En producción: broker durable, reintentos, dead-letter y consumidores idempotentes.
Facturación	Sistema legado aislado por adaptador.	Contrato de integración, timeout, traducción de códigos, pruebas de compatibilidad y conciliación.

Pruebas automatizadas incluidas

Prueba	Aserción que protege
Catálogo disponible	`GET /api/v1/restaurantes` responde 200 y lista `R001`.
Flujo feliz hasta asignación	Crear → pagar → aceptar → asignar; estado final de ese escenario `REPARTIDOR_ASIGNADO` e id de repartidor no vacío.
Pago rechazado	Token `tok_demo_rechazado` produce `CANCELADO` y `estadoPago=RECHAZADO`.

Pruebas manuales prioritarias con requests.http

- Rechazo del restaurante: pago aprobado → `aceptado=false`; comprobar CANCELADO y REEMBOLSADO.
- Tiempo agotado: aprobar pago y no emitir decisión; revisar después de cinco minutos + intervalo del scheduler.
- Seguimiento: abrir SSE y después actualizar ubicación; observar el evento `UBICACION_ACTUALIZADA` en el cliente HTTP.
- Entrega: confirmar con el repartidor asignado, enviar calificación, pedir factura y comprobar que el adaptador no representa una conexión fiscal real.

Limitación de validación en esta sesión: la ejecución de Maven no se pudo completar en el entorno de trabajo porque DNS no resolvió `repo.maven.apache.org`; por eso las pruebas están implementadas, pero deben ejecutarse localmente con `./mvnw test` o `mvnw.cmd test`.

11 · DEFENSA

Guion para el debate de cinco minutos

Guion propuesto (aprox. 5 minutos)

"Nuestro diseño se llama RapiEntrega y responde al caso de la unidad: una plataforma que conecta clientes, restaurantes y repartidores, cobra mediante una pasarela externa y conserva un sistema de facturación de 2012. Partimos del proceso de negocio y no de los controladores que queríamos programar. Primero descompusimos el pedido en acciones pequeñas; después descartamos detalles de interfaz que no son servicios de negocio; identificamos entidades como Restaurante, Producto, Pedido, Repartidor y Factura; aislamos la lógica de proceso de pago, plazo, cancelación y asignación; y finalmente definimos los recursos REST con rutas, métodos, JSON y errores.

La matriz asigna una capacidad propietaria por actividad. El catálogo es dueño del menú y precios; el proceso de pedidos calcula el total y controla las transiciones; pagos encapsula el proveedor; logística elige repartidor y actualiza ubicación; seguimiento retransmite eventos; reputación registra calificación; facturación adapta la interfaz heredada. Así evitamos que cualquier servicio repita la misma regla o acceda directamente a datos que no le pertenecen.

Aplicamos patrones del catálogo de Erl. Process Abstraction concentra la coordinación del proceso sin mover el catálogo o el pago dentro de la misma responsabilidad. Service Façade y Legacy Wrapper aíslan los contratos internos de dos dependencias externas muy diferentes: la pasarela de tarjetas y el sistema fiscal heredado. Event-Driven Messaging permite publicar cambios del pedido y que el seguimiento los consuma sin bloquear el cambio de estado. Compensating Service Transaction representa el caso en que el dinero se aprobó pero el restaurante rechaza o vence la ventana de cinco minutos: se cancela el pedido y se solicita el reembolso. Asynchronous Queuing se propone como evolución para hacer durables los eventos cuando la plataforma se distribuya; no afirmamos que el bus local de Spring sea una cola durable.

Para las interacciones que requieren una respuesta inmediata, como consultar menú, crear pedido o conocer el resultado inicial del pago, usamos REST request-response. Para los cambios a los que pueden suscribirse más consumidores, como estado de pedido y ubicación, usamos publicación-suscripción y un canal SSE unidireccional hacia el cliente. La asignación es síncrona en el prototipo porque necesitamos confirmar que se reservó un repartidor; en producción la reserva debe ser atómica y puede formar parte de un proceso asíncrono con reintentos.

La solución que entregamos es una aplicación Spring Boot modular, con contratos por servicio, OpenAPI, DTO validados, manejo uniforme de errores y pruebas del catálogo, pago rechazado y camino feliz hasta la asignación. No confundimos una demostración académica con producción: almacenamiento, pago y facturación son simulados, no hay autenticación y los eventos no se retienen en un broker durable. El siguiente paso sería base de datos persistente, idempotencia en pagos, autorización por rol, broker, observabilidad y pruebas de carga. La ventaja central de SOA aquí es poder cambiar o escalar una capacidad sin reescribir todo el proceso."

Preguntas probables y respuestas cortas

Pregunta del equipo / docente	Respuesta defendible
¿Por qué no microservicios desde el inicio?	La estructura separa dominios y contratos, pero el monolito modular baja la complejidad de la práctica. La extracción por dominio exige persistencia y mensajería compartida confiables.
¿ApplicationEventPublisher es una cola?	No. Es un bus local dentro del proceso. La solución usa pub/sub asíncrono para demostrar el patrón; para durabilidad y varios nodos debe sustituirse o complementarse con un broker.
¿Qué sucede si el pago se aprueba y el restaurante no contesta?	El temporizador expira a los cinco minutos, el pedido se cancela y se solicita reembolso como transacción compensatoria. Se requiere conciliación operativa si el reembolso falla.
¿Cómo se conserva el sistema de facturación de 2012?	A través de un puerto y un Legacy Wrapper que traduce la solicitud y respuesta. El código suministrado es un simulador y no pretende conectarse al sistema real.
¿Cómo se evita que el usuario cambie el precio?	La solicitud envía IDs de producto y cantidades; el servidor consulta el precio del catálogo y calcula subtotal y total.

Referencias

[1] "Arquitectura Orientada a Servicios · Retos · Unidad 2: Diseño de servicios", PDF de la actividad proporcionado en clase, Reto 2, pp. 10–17.

[2] Erl, Thomas. Service-Oriented Architecture: Analysis & Design for Services and Microservices, 2.ª ed., capítulo 7 "REST Service Modeling Process". Índice oficial: https://www.thomaserl.com/book/service-oriented-architecture-analysis-design-for-services-and-microservices-second-edition-2/table-of-contents/index.html

[3] Erl, Thomas (ed.). SOA Design Patterns. Prentice Hall, 2009. Catálogo y capítulos: https://www.thomaserl.com/book/soa-design-patterns/table-of-contents/index.html